BYZANTINE-RESILIENT FEDERATED LEARNING FOR TACTICAL UAV SWARMS OVER LOSSY WIRELESS LINKS

Project report submitted to the
National Institute of Technology Warangal
For award of the degree

of

Bachelor of Technology

in Electronics and Communication Engineering

by

Nasika Sasank	**23ECB0A03**
Aaron George	**23ECB0F06**
Sachin Sreekumar Nair	**23ECB0F11**

Under the guidance of

Dr. Chayan Bhar



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

November 2026

APPROVAL OF THE VIVA-VOCE BOARD

DD/MM/YYYY

Certified that the project report entitled BYZANTINE-RESILIENT FEDERATED LEARNING FOR TACTICAL UAV SWARMS OVER LOSSY WIRELESS LINKS submitted by NASIKA SASANK (23ECB0A03), AARON GEORGE (23ECB0F06), SACHIN SREEKUMAR NAIR (23ECB0F11) to the National Institute of Technology, Warangal, for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering has been accepted by the examiners and that the students have successfully defended the work in the viva-voce examination held today.

(Examiner) (Examiner) (Examiner)

(Supervisor) (Head of the Department)

(Chairman)

(The certificate is to be printed on the Departmental Letter-Head)

CERTIFICATE

This is to certify that the project report entitled **Byzantine-Resilient Federated Learning for Tactical UAV Swarms over Lossy Wireless Links**, submitted by **Nasika Sasank** (23ECB0A03), **Aaron George** (23ECB0F06), **Sachin Sreekumar Nair** (23ECB0F11) to the National Institute of Technology Warangal, is a record of bona fide work carried out under my supervision in the Department of Electronics and Communication Engineering during the academic year 2026–2027, and we consider it worthy of consideration for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering of the Institute.

Dr. Chayan Bhar	Prof. Ravi Kumar Jatoth
Supervisor	Head of the Department

Date:

DECLARATION

We certify that

a. The work contained in this report is original and has been done by us under the general supervision of our supervisor.

b. The work has not been submitted to any other Institute for any degree or diploma.

c. We have followed the guidelines provided by the Institute in writing the report.

d. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.

e. Whenever we have quoted written materials from other sources, we have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Nasika Sasank	Aaron George	Sachin Sreekumar Nair
23ECB0A03	23ECB0F06	23ECB0F11

Signature of the Students

Acknowledgements

We thank our supervisor, Dr. Chayan Bhar, for steering this project towards realistic wireless conditions and fair, like-for-like comparisons, and for asking us to state our findings with the evidence behind them. We thank the Head of the Department, Prof. Ravi Kumar Jatoth, and the faculty of the Department of Electronics and Communication Engineering for their support. We also thank the creators of the DeepSense 6G dataset for making their UAV measurements available.

Nasika Sasank

Aaron George

Sachin Sreekumar Nair

Abstract

Swarms of unmanned aerial vehicles (UAVs) can learn a shared perception model with federated learning (FL): each UAV trains on the images it collects and sends only a model update to a ground base station. In a tactical setting two problems arise together. Air-to-ground links lose packets because of fading, distance, handover and jamming, and some UAVs may be captured and made to send poisoned updates. Most Byzantine-robust aggregation rules assume that every update arrives intact, while most work on FL over unreliable links assumes that every client is honest. This project studies both problems in one system.

We built a simulation framework in PyTorch for hierarchical FL over a UAV swarm. Ten UAVs with non-identically distributed aerial images are served by three base stations, which aggregate locally and synchronize every five rounds. Uplink packet losses come either from a synthetic channel with path loss, Rayleigh fading and a ground jammer, or from a measured UAV received-power trace (DeepSense 6G Scenario 23) replayed at a chosen signal-to-noise ratio. Three of the UAVs mount sign-flip, ALIE-style or inner-product-manipulation attacks. The framework includes FedAvg, coordinate-wise median, FLTrust, a cosine-weighted rule taken from prior UAV-FL work, and a reputation-based aggregator, CCRA, that we designed. CCRA compares each update with a reference update computed on a small trusted dataset, keeps a reputation for every UAV across rounds, and can use link information in two ways: by ignoring coordinates whose packets were lost (erasure masking), and by letting link telemetry slow down how fast a UAV regains trust.

The work proceeded in three phases. A FedAvg baseline reached 62.6% (IID) and 60.2% (non-IID) test accuracy. A preliminary single-seed study suggested that CCRA resisted a label-flipping attack better than the cosine-weighted baseline. To test which

part of CCRA was responsible, we then ran a controlled, prespecified study of 720 training runs on an A100 GPU: 10 paired seeds, four attack settings, three channel conditions and six aggregators. Diverged runs were scored at chance rather than discarded, and all uncertainty was estimated with a paired bootstrap over seeds.

The study gave a clear and partly negative answer. On a weak (5 dB) link under attack, erasure masking improved accuracy by 4.2 percentage points (95% confidence interval [2.2, 5.9]). Adding link telemetry to the reputation update changed accuracy by -3.5 points ([−5.1, −2.1]) because it rejected 41.2% of honest updates, against 13.8% without telemetry. We explain this with a short analysis of the reputation rule: when rewards are slowed but penalties are not, an honest UAV's reputation settles near its worst rounds. All reference-based aggregators avoided divergence, whereas FedAvg and the median diverged in 41 and 26 of 120 runs.

The result changes the direction of the project. The useful channel-aware component is erasure masking, and the reputation reward rule must be redesigned before link telemetry can help. The findings are written up as a four-page letter in IEEE format. The second phase will revise the reward rule, test it on fresh seeds and higher-mobility traces, and evaluate adaptive attackers.

Index Terms : Federated learning, UAV swarm, Byzantine robustness, packet erasure, hierarchical aggregation, reputation.

Contents

Certificate of Approval ii

Certificate iii

Declaration iv

Acknowledgements v

Abstract vi

List of Figures xi

List of Tables xii

List of Abbreviations xiii

List of Symbols xiv

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Aim and Scope 2

1.4 Objectives 3

1.5 Contributions 3

1.6 Organization of the Report . . . 3

2 Literature Review **5**

2.1 Federated Learning . . . 5

2.2 Byzantine-Robust Aggregation . . . 5

2.3 Federated Learning over Unreliable Wireless Links . . . 6

2.4 Federated Learning for UAV Swarms . . . 7

2.5 Research Gap . . . 7

3 Theoretical Study **8**

3.1 Hierarchical Federated Learning for a UAV Swarm . . . 8

3.1.1 Dataset and Model . . . 9

3.2 Wireless Uplink Models . . . 9

3.2.1 Synthetic Jammed Channel . . . 10

3.2.2 Measured-Trace Channel . . . 10

3.2.3 Packet Erasures . . . 11

3.3 Threat Model . . . 11

3.4 Aggregation Rules . . . 12

3.4.1 Baselines . . . 12

3.4.2 Context-Calibrated Robust Aggregation (CCRA) . . . 12

3.4.3 Ablation Variants . . . 13

3.4.4 Analysis of the Reputation Update . . . 14

4 Experimental Study **15**

4.1 Implementation and Verification . . . 15

4.2 Experimental Protocol . . . 16

4.2.1 Preliminary Study 16
4.2.2 Controlled Ablation Study 16
4.3 FedAvg Baseline Results 17
4.4 Preliminary Study Results 17
4.5 Controlled Ablation Study Results 19
4.5.1 Stability and Overall Accuracy 19
4.5.2 Which Kind of Channel Awareness Helps 20
4.5.3 Mechanism: Who the Gate Rejects 21
4.5.4 Comparison with FLTrust 23
4.6 Discussion 23
4.6.1 What the Results Show 23
4.6.2 Why a Negative Result Is Useful 23
4.6.3 Threats to Validity 24

5 Conclusions and Future Scope of Study 25
5.1 Summary of the Work 25
5.2 Conclusions 25
5.3 Scope for Future Work (Phase II) 26

References 28

Appendix-A 30

Appendix-B 31

List of Figures

3.1 Hierarchical FL over a UAV swarm. Each base station aggregates the updates of the UAVs it serves, some of which arrive with lost packets; edge models are synchronized every E rounds. 8

4.1 Test accuracy and loss of the FedAvg baseline over 50 rounds. 17

4.2 Preliminary study: final accuracy in the four scenarios. 18

4.3 Preliminary study: aggregation weight given to attackers and to jammed honest UAVs under label flipping on the jammed channel. 19

4.4 Final accuracy under attack (mean over sign flip, ALIE and IPM and 10 seeds), with 95% bootstrap confidence intervals. 20

4.5 Paired accuracy differences under attack with 95% bootstrap intervals. The grey band is the prespecified ± 1 point region of practical equivalence. . . . 22

4.6 Gate decisions under attack at 5 dB, as a percentage of received updates, averaged over attacks and seeds. 22

List of Tables

2.1 Coverage of the problem in the reviewed work 7

4.1 FedAvg baseline accuracy after 50 rounds 17

4.2 Preliminary study: final accuracy (%), single seed, 30 rounds 18

4.3 Outcomes over all 720 runs (120 per method). Diverged runs count as 10%. 19

4.4 Mean final accuracy (%) by attack, method and channel. Best value in each block and column in bold. 21

A.1 Settings of the controlled study 30

B.1 Repository layout 31

List of Abbreviations

ALIE	A Little Is Enough (model-poisoning attack)
BS	Base station
CCRA	Context-Calibrated Robust Aggregation (the aggregator studied here)
CI	Confidence interval
CNN	Convolutional neural network
CosAvg	Cosine-similarity weighted averaging
EMA	Exponential moving average
EW	Electronic warfare
FedAvg	Federated averaging
FL	Federated learning
HFL	Hierarchical federated learning
IID	Independent and identically distributed
IPM	Inner-product manipulation (model-poisoning attack)
PDR	Packet delivery ratio
PER	Packet error rate
pp	Percentage points
RMS	Root mean square
SGD	Stochastic gradient descent
SINR	Signal-to-interference-plus-noise ratio
SNR	Signal-to-noise ratio
TEE	Trusted execution environment
UAV	Unmanned aerial vehicle

List of Symbols

K	Number of UAVs (clients)
B	Number of base stations
E	Edge rounds between global synchronizations
P	Number of model parameters
L	Parameters per packet
f	Number of Byzantine UAVs
$\mathcal{D}_k$, n_k	Local dataset of UAV k and its size
$\mathcal{D}_0$	Trusted root dataset held by each base station
$\mathbf{w}^t$, $\mathbf{w}_b^t$	Global model and edge model of BS b in round t
$\boldsymbol{\Delta}_k^t$	Model update sent by UAV k in round t
$\mathbf{m}_k$	Delivery mask: 1 where the packet carrying a coordinate arrived
$\mathbf{r}$	Reference update computed on $\mathcal{D}_0$
γ_k, $\bar{\gamma}$	Uplink SNR of UAV k; reference SNR of the trace
p_k	Packet error probability of UAV k
a, γ_{th}	Slope and threshold of the packet-error curve
s_k	Norm-clipping factor
c_k	Masked cosine similarity to the reference update
b_k	Behaviour score in $[0, 1]$
T_k	Reputation (trust) of UAV k
e_k	Link evidence in $[0, 1]$ that slows reputation rewards
β, τ	Reputation memory and exclusion gate
ω_k	Aggregation weight of UAV k
α	Dirichlet concentration of the non-IID data split
λ, z, ϵ	Strengths of the sign-flip, ALIE and IPM attacks

Chapter 1

Introduction

1.1 Background

Small unmanned aerial vehicles (UAVs) are now used in groups, or swarms, for surveillance, search and rescue, disaster assessment and border patrol. Each UAV carries a camera and collects images of the area it covers. A perception model trained on all of these images is more accurate than one trained on any single UAV's data, but sending raw imagery to a ground station is costly in bandwidth and exposes sensitive data if a link is intercepted.

Federated learning (FL) addresses this by moving training to the data. Each UAV trains a copy of the model on its own images and uploads only the resulting model update. A server (here, a ground base station) combines the updates into a new global model and sends it back. The raw images never leave the UAV, and an update is usually much smaller than the data it was computed from.

A tactical swarm, however, breaks two assumptions that standard FL relies on.

1. **Links are unreliable.** Air-to-ground channels fade as the UAV moves, hand over between base stations, and may be deliberately jammed. Parts of an update are lost, and some updates miss the round deadline entirely.

2. **Some clients may be hostile.** A captured or spoofed UAV can follow the protocol while sending a poisoned update designed to damage the global model. Such clients are called *Byzantine*.

Most Byzantine-robust aggregation rules assume that every update arrives intact, and most work on FL over imperfect channels assumes that every client is honest. A swarm needs both at once.

1.2 Problem Statement

Given a UAV swarm that trains a shared image classifier with hierarchical FL over lossy air-to-ground links, in which a minority of UAVs are Byzantine, design and evaluate an aggregation rule that (i) keeps the global model accurate and stable, (ii) does not penalize honest UAVs simply because their links are poor, and (iii) can be compared fairly against established rules under identical conditions.

1.3 Aim and Scope

The aim of this project is to determine *which* kind of channel awareness actually helps a robust aggregator in a UAV swarm, and to quantify the effect with a controlled, reproducible experiment. Its scope in Phase I is:

- a simulation of hierarchical FL with up to three base stations, UAV association and handover;
- packet-level uplink losses from a synthetic jammed channel and from a measured UAV received-power trace;
- four model-poisoning and data-poisoning attacks;
- six aggregation rules evaluated on the same data, models, attackers and channel realizations.

Hardware flight tests, real radio transmission and attacks that forge link telemetry are outside the Phase I scope.

1.4 Objectives

1. Build an FL baseline for a UAV swarm on an aerial image dataset and measure its accuracy under IID and non-IID data.
2. Model a lossy uplink in which lost packets erase parts of an update, and add Byzantine attackers.
3. Design a reputation-based aggregator (CCRA) that uses a trusted reference update, and give it two forms of channel awareness: erasure masking and telemetry-weighted reputation.
4. Separate the effect of each component with a prespecified, multi-seed ablation study, and report the results with honest uncertainty estimates.

1.5 Contributions

- An open, tested PyTorch framework for hierarchical UAV-swarm FL with trace-driven packet erasures, handover and five attacks (43 automated tests).
- A preliminary study, then a controlled 720-run study whose analysis plan and decision threshold were fixed before the runs.
- The finding that erasure masking improves accuracy under attack on weak links by 4.2 percentage points, while telemetry-weighted reputation lowers it, together with an analysis of the reputation rule that explains why.
- A four-page manuscript in IEEE letter format that reports these results.

1.6 Organization of the Report

Chapter 2 reviews federated learning, Byzantine-robust aggregation and FL over unreliable wireless links, and identifies the gap this project addresses. Chapter 3 presents the theoretical study: the system and threat models, the aggregation rules and an analysis

of the reputation update. Chapter 4 presents the experimental study: implementation, protocol, the baseline, the preliminary study and the controlled study, and a discussion of what they show. Chapter 5 states the conclusions and the plan for Phase II. The appendices, after the references, list the experimental settings and the steps needed to reproduce every result.

Chapter 2

Literature Review

2.1 Federated Learning

McMahan et al. [2017] introduced federated averaging (FedAvg). In each round the server sends the global model to a set of clients, each client runs a few epochs of stochastic gradient descent on its local data, and the server replaces the global model by the average of the returned models, weighted by local dataset size. FedAvg works well when client data are independent and identically distributed (IID). When each client sees a skewed mix of classes, local models drift apart and accuracy falls. Hsu et al. [2019] proposed sampling each client's class proportions from a Dirichlet distribution with concentration α, which has become the standard way to control the degree of non-IID skew in experiments; we use $\alpha = 0.5$.

Hierarchical FL adds an intermediate layer of edge servers. Clients send updates to a nearby edge server, which aggregates them frequently, and the edge servers synchronize with each other less often. This matches a UAV swarm served by several ground base stations, each covering part of the operating area.

2.2 Byzantine-Robust Aggregation

A single malicious client can move the FedAvg average arbitrarily far, so a large body of work replaces the mean by a robust statistic.

- **Krum** [Blanchard et al., 2017] selects the update whose summed distance to its nearest neighbours is smallest. It requires $n \geq 2f + 3$ clients to tolerate f attackers.
- **Coordinate-wise median and trimmed mean** [Yin et al., 2018] take the median, or a trimmed mean, of each parameter separately, and come with statistical error guarantees.
- **FLTrust** [Cao et al., 2021] gives the server a small clean root dataset. The server computes its own update on that data each round, scores each client by the (clipped at zero) cosine similarity to it, and rescales every client update to the norm of the server update before averaging.

These rules are evaluated against attacks of increasing strength. A sign-flip attack sends a scaled negative of an honest update. ALIE ("A Little Is Enough") [Baruch et al., 2019] shifts each coordinate by a small multiple of the honest standard deviation, which is enough to evade distance-based rules. Inner-product manipulation (IPM) [Xie et al., 2019] sends a negative multiple of the honest mean so that the aggregate's inner product with the true gradient becomes negative.

Reputation-based defences keep a trust score for each client across rounds, so that an attacker caught once stays down-weighted. They suit swarms, whose members persist for a whole mission, but recent work shows that adaptive attackers can learn to evade them [Liu et al., 2026].

2.3 Federated Learning over Unreliable Wireless Links

In wireless FL the update itself travels over a noisy channel. Shirvanimoghaddam et al. [2022] study FL in which the communication links are erroneous; this letter is the base paper for our study of lossy links. Barkatsa et al. [2024] address jamming attacks on wireless FL networks with a Bayesian-game formulation. Li et al. [2022] combine communication efficiency, differential privacy and Byzantine robustness. In each case the emphasis is on either the channel or the adversary, not on how lost packets interact with a robust aggregator.

2.4 Federated Learning for UAV Swarms

Xiao et al. [2021] propose fully decentralized FL for on-board missions of a UAV swarm. Liao et al. [2023] propose UAV-assisted FL based on a trusted execution environment (TEE) and aggregate updates by their cosine similarity to the mean update (CosAvg); our preliminary study re-implements CosAvg as a baseline. Realistic UAV link data have recently become available: the DeepSense 6G dataset [Alkhateeb et al., 2023] includes a scenario with a flying UAV and a ground base station [Charan and Alkhateeb, 2025], which we use to drive packet losses in simulation.

2.5 Research Gap

Table 2.1 summarizes the gap. Robust rules either compare updates as if they were complete, treating a lost packet as a zero, or ignore the channel entirely. Channel-aware designs do not model adversarial clients. It is also natural to assume that link telemetry (signal quality, delivery ratio, handover state) should make a robust aggregator *fairer* to honest UAVs on poor links, but we found no controlled test of that assumption. This project provides one.

Table 2.1 Coverage of the problem in the reviewed work. Entries for the letters [Barkatsa et al., 2024, Shirvanimoghaddam et al., 2022, Xiao et al., 2021] and for [Liao et al., 2023] reflect their titles and our reading so far and are to be confirmed against the full texts.

Work	Byzantine clients	Packet loss	UAV setting	Measured channel
FedAvg [McMahan et al., 2017]	–	–	–	–
Krum, median [Blanchard et al., 2017, Yin et al., 2018]	✓	–	–	–
FLTrust [Cao et al., 2021]	✓	–	–	–
Erroneous links [Shirvanimoghaddam et al., 2022]	–	✓	–	–
Jamming mitigation [Barkatsa et al., 2024]	–	✓	–	–
UAV-swarm FL [Xiao et al., 2021]	–	–	✓	–
TEE + CosAvg [Liao et al., 2023]	✓	–	✓	–
This project	✓	✓	✓	✓

Chapter 3

Theoretical Study

This chapter describes the simulated system, the attacker and the aggregation rules compared, and analyses the reputation update. Chapter 4 describes how they were evaluated. All of it is implemented in the project repository; file names are given where they help locate the code.

3.1 Hierarchical Federated Learning for a UAV Swarm

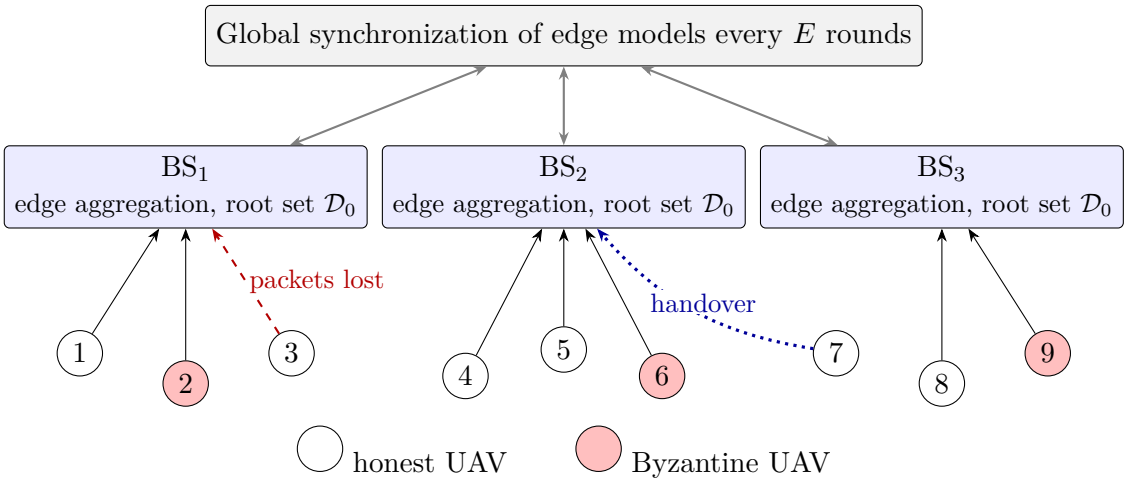


Figure 3.1 Hierarchical FL over a UAV swarm. Each base station aggregates the updates of the UAVs it serves, some of which arrive with lost packets; edge models are synchronized every E rounds.

Figure 3.1 shows the system. K UAVs are served by B ground base stations (BSs). Each UAV k holds a private dataset $\mathcal{D}_k$ of n_k images. In every round t:

1. each UAV attached to BS b starts from that BS's edge model $\mathbf{w}_b^t$ and trains it for

one local epoch of SGD on $\mathcal{D}_k$;

2. it uploads the update $\mathbf{\Delta}_k^t$ (new local model minus the edge model), which is split into packets of L parameters;

3. the BS aggregates the updates it receives and sets $\mathbf{w}_b^{t+1} = \mathbf{w}_b^t + \mathrm{Agg}(\cdot)$;

4. every E rounds the edge models are averaged, weighted by the number of training samples held by the UAVs in each cell, and the result is sent back to every BS.

UAVs attach to the BS with the strongest received power. A UAV hands over only when another BS is stronger by a hysteresis margin of 3 dB for two consecutive rounds, which avoids ping-pong handovers (`fl/topology.py`).

3.1.1 Dataset and Model

Images come from CIFAR-10, modified to resemble aerial imagery (`fl/data.py`). Each image is rotated by a random multiple of 90°, since a downward-looking camera has no preferred orientation, and with probability 0.3 or 0.2 is down-sampled to 24×24 or 16×16 pixels and up-sampled again to imitate higher flight altitudes. The 50,000 training images are split among the UAVs either uniformly (IID) or with Dirichlet(α) class proportions; 500 images (50 per class) are held out as the trusted root set $\mathcal{D}_0$ and are never given to any UAV.

The classifier, TacticalEdgeCNN, is a small convolutional network with about 156k parameters and group normalization, chosen so that it could plausibly be trained on board. Group normalization is used instead of batch normalization because batch statistics do not average well across clients with different class mixes.

3.2 Wireless Uplink Models

The channel decides, for every packet of every update, whether it arrives (`fl/channel.py`). Two models are used.

3.2.1 Synthetic Jammed Channel

UAVs move by a random walk at 100 m altitude over an 800×800 m area. The received power from UAV k at its serving BS is

$$P_{\mathrm{rx},k} = P_{\mathrm{tx}} - \mathrm{PL}(d_k) + 10\log_{10}|h_k|^2, \qquad \mathrm{PL}(d) = 40 + 10\eta\log_{10} d \text{ dB}, \tag{3.1}$$

with $P_{\mathrm{tx}} = 20$ dBm, path-loss exponent $\eta = 2.2$, and Rayleigh fading $|h_k|^2 \sim \mathrm{Exp}(1)$. A ground jammer of power 10 dBm targets a sector of the swarm, and the SINR is

$$\gamma_k = P_{\mathrm{rx},k} - 10\log_{10}\big(N_0 + J_k\big), \tag{3.2}$$

where N_0 corresponds to -90 dBm and J_k is the jamming power received at UAV k's link (zero for UAVs outside the jammed sector).

3.2.2 Measured-Trace Channel

For realism we replay a measured UAV link from DeepSense 6G Scenario 23 [Alkhateeb et al., 2023, Charan and Alkhateeb, 2025], in which a drone flies around a ground receiver that records 60 GHz beam powers together with position, altitude and speed. The dataset's power values are uncalibrated, so only their variation relative to the median is used:

$$\gamma_k = \bar{\gamma} + \big(P_k^{\mathrm{meas}} - \mathrm{med}\, P^{\mathrm{meas}}\big) - 10\eta\log_{10}\frac{d_{k,b}}{d_{k,1}}, \tag{3.3}$$

where $\bar{\gamma}$ is an assumed reference SNR and the last term adjusts the measured link to the distance of a virtual BS b. Each UAV replays a different flight segment. Two settings are used: $\bar{\gamma} = 15$ dB, a good link, and $\bar{\gamma} = 5$ dB, a weak link. They replay identical trajectories, so only the link quality differs.

3.2.3 Packet Erasures

In both models each packet is lost independently with probability

$$p_k = \frac{1}{1 + \exp\big(a(\gamma_k - \gamma_{\text{th}})\big)}, \tag{3.4}$$

with $a = 1$ and $\gamma_{\text{th}} = 3\,\text{dB}$. An update whose transmission time at the Shannon rate exceeds the 2 s round deadline is dropped entirely. The BS therefore receives $\tilde{\boldsymbol{\Delta}}_k = \mathbf{m}_k \odot \boldsymbol{\Delta}_k$, where the mask $\mathbf{m}_k$ is 1 for delivered coordinates and 0 elsewhere. The BS also records link telemetry for each UAV: SNR, packet delivery ratio (PDR), long-run reliability, distance, speed and handover state.

3.3 Threat Model

A fixed set of $f = 3$ of the $K = 10$ UAVs is Byzantine. They follow the protocol (they train, transmit on time and use the same channel) but replace their update with one of the following (`fl/attacks.py`, `fl/client.py`):

- **Label flip** (data poisoning): train on labels mapped $c \mapsto 9 - c$.
- **Sign flip**: send $-\lambda\boldsymbol{\Delta}_k$ with $\lambda = 5$.
- **ALIE-style** [Baruch et al., 2019]: send $\boldsymbol{\mu} - z\,\boldsymbol{\sigma} \odot \text{sign}(\boldsymbol{\mu})$ with $z = 1.5$, where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are the mean and standard deviation of the honest updates in the attacker's cell.
- **IPM** [Xie et al., 2019]: send $-\epsilon\boldsymbol{\mu}$ with $\epsilon = 0.5$.

The ALIE and IPM attackers know the honest updates of their cell, which is a strong assumption chosen to stress the defences. The BS's telemetry is assumed authentic; attacks that forge it are left for Phase II.

3.4 Aggregation Rules

All rules receive exactly the same inputs: the received updates, their masks, sample counts and telemetry (`fl/aggregators.py`).

3.4.1 Baselines

FedAvg [McMahan et al., 2017] averages the received updates weighted by n_k, with lost coordinates counted as zeros. **CosAvg**, the aggregation of the earlier UAV-FL work [Liao et al., 2023] as re-implemented here, weights each update by $\max(0, \cos(\boldsymbol{\Delta}_k, \bar{\boldsymbol{\Delta}}))\, n_k$, where $\bar{\boldsymbol{\Delta}}$ is the FedAvg mean. **Coordinate-wise median** [Yin et al., 2018] takes the median of the delivered values of each parameter. **FLTrust** [Cao et al., 2021] computes a root update $\mathbf{r}$ on $\mathcal{D}_0$ (with as many local steps as a typical UAV), weights each UAV by $\max(0, \cos)$ to $\mathbf{r}$ on delivered coordinates, and rescales each update to the norm of $\mathbf{r}$.

3.4.2 Context-Calibrated Robust Aggregation (CCRA)

CCRA is the aggregator designed in this project. Each BS applies the following steps every round.

1. **Reference update.** Compute $\mathbf{r}$ on $\mathcal{D}_0$, as in FLTrust. Our diagnostics showed that, under non-IID data, the mean or median of the UAV updates is not a reliable reference: attackers and honest UAVs with skewed data had similar cosine similarity to it.

2. **Norm clipping.** Let ρ_k be the RMS of the delivered entries of $\tilde{\boldsymbol{\Delta}}_k$ and scale the update by $s_k = \min(1,\ \mathrm{med}_j \rho_j / \rho_k)$, which defeats scaling attacks.

3. **Masked similarity.** $c_k = \cos\big(\mathcal{S}(s_k \tilde{\boldsymbol{\Delta}}_k; \mathbf{m}_k),\ \mathcal{S}(\mathbf{r}; \mathbf{m}_k)\big)$, where $\mathcal{S}(\cdot; \mathbf{m})$ keeps only the coordinates with $\mathbf{m} = 1$. Lost packets therefore do not make an honest update look dissimilar.

4. **Behaviour score.** $b_k = \mathrm{clip}\big(c_k / \mathrm{med}_{j: c_j > 0} c_j,\ 0,\ 1\big)$. Updates pointing away from the reference get $b_k = 0$.

5. **Reputation.** With memory β, initial trust 1 and evidence $e_k \in [0, 1]$,

$$T_k \leftarrow T_k + \eta_k(b_k - T_k), \qquad \eta_k = \begin{cases} (1-\beta)\, e_k, & b_k > T_k, \\ 1-\beta, & b_k \le T_k. \end{cases} \tag{3.5}$$

   Penalties apply at full rate, so an attacker cannot hide by dropping its own packets; only rewards are slowed on poor links.

6. **Gating and averaging.** UAVs with $T_k < \tau$ are excluded. The others get weight $\omega_k = n_k T_k$, discounted for staleness, and each coordinate is averaged only over the UAVs that delivered it:

$$\mathrm{Agg}_p = \frac{\sum_k \omega_k\, m_{k,p}\, s_k \tilde{\Delta}_{k,p}}{\sum_k \omega_k\, m_{k,p}}. \tag{3.6}$$

The evidence term multiplies five link factors:

$$e_k = \mathrm{PDR}_k \cdot R_k \cdot h_k \cdot \frac{1}{1 + d_k/A} \cdot \frac{1}{1 + |v_k|}, \tag{3.7}$$

namely the packet delivery ratio, long-run reliability, a handover factor ($h_k = 0.5$ during a handover, 1 otherwise), a distance factor with A the area size, and a normalized-speed factor.

3.4.3 Ablation Variants

To find out which part of CCRA matters, three variants are compared.

- **CCRA (full)**: all steps, with evidence e_k from telemetry.
- **Mask-only**: $e_k = 1$, but the masks are used in steps 2–6.
- **No-context**: $e_k = 1$ and $\mathbf{m}_k = \mathbf{1}$, so lost coordinates enter the computation as zeros, as a channel-unaware server would see them.

The code also has an option to correct c_k for the noise expected at the reported SINR. It is inactive in the controlled study, which adds no residual noise to delivered parameters, so the full-versus-mask-only contrast isolates e_k.

3.4.4 Analysis of the Reputation Update

The asymmetric update (3.5) has a consequence that was not intended. Suppose a UAV's evidence is constant, $e_k = e$, and its scores b are drawn independently from some distribution. The expected change in reputation is

$$\mathbb{E}[\Delta T] = (1-\beta)\Big(e\,\mathbb{E}\big[(b-T)^+\big] - \mathbb{E}\big[(T-b)^+\big]\Big), \tag{3.8}$$

so the reputation settles where

$$e\,\mathbb{E}\big[(b-T^\star)^+\big] = \mathbb{E}\big[(T^\star-b)^+\big]. \tag{3.9}$$

This steady state is characterized as follows.

Proposition 3.1. Let $b \in [0,1]$ be i.i.d. with $\mathbb{E}[b] > 0$ and let $g_e(T) = e\,\mathbb{E}[(b-T)^+] - \mathbb{E}[(T-b)^+]$. For every $e \in (0,1]$, g_e has a unique root $T^\star(e) \in (0,1)$. Moreover $T^\star(1) = \mathbb{E}[b]$, $T^\star(e)$ is increasing in e, and $T^\star(e) \to \operatorname{ess\,inf} b$ as $e \to 0$.

Proof. g_e is continuous and strictly decreasing in T, with $g_e(0) = e\,\mathbb{E}[b] > 0$ and $g_e(1) = -\mathbb{E}[1-b] < 0$, so it has exactly one root in $(0,1)$. For $e = 1$, $g_1(T) = \mathbb{E}[b] - T$. Because $\partial g_e/\partial e = \mathbb{E}[(b-T)^+] \geq 0$ while g_e decreases in T, the root rises with e. Finally, as $e \to 0$, $g_e(T) \to -\mathbb{E}[(T-b)^+]$, which is negative for every $T > \operatorname{ess\,inf} b$, while $g_e(T) > 0$ for every $T < \operatorname{ess\,inf} b$. ■

In words: with full evidence the reputation settles at the mean score; as the evidence falls, it moves down towards the lowest score the UAV ever receives. An honest UAV with skewed data has noisy scores, so on a weak link its reputation tracks its worst rounds and falls below the gate τ more often. An attacker's scores are already near zero, so slowing its rewards adds little. Section 4.5.3 tests this prediction against the measured gate decisions.

Chapter 4

Experimental Study

This chapter describes how the framework was implemented and verified, the protocol of each experiment, and the results, in the order the work was done: the FedAvg baseline, the preliminary single-seed study, and the controlled 720-run study. Every number in Section 4.5 is generated from the committed run-level data by the script described in Appendix B.

4.1 Implementation and Verification

The framework is written in Python with PyTorch. Runs are configured by a single dataclass, and every run stores its full configuration, per-round history and final model. The repository contains 43 automated tests covering data partitioning, the FedAvg update, attack construction, every robust aggregator, the hierarchical server, handover logic, trace loading and the analysis code. A code review of the hierarchical extension found three problems, which were fixed before the controlled study was run: the reputation rule shielded jammed attackers, the no-context ablation still read packet delivery ratios, and FLTrust's root update took too few training steps.

4.2 Experimental Protocol

4.2.1 Preliminary Study

The first comparison used the synthetic jammed channel with one BS, a 30% label-flip attack, 30 rounds and a single seed. It compared FedAvg, CosAvg and an early version of CCRA under four scenarios: no attack or attack, each with an ideal or a jammed lossy channel. Its purpose was to check that the pipeline worked and to estimate effect sizes.

4.2.2 Controlled Ablation Study

The main study used a full factorial design: six aggregators (FedAvg, median, FLTrust and the three CCRA variants), four attack settings (clean, sign flip, ALIE, IPM), three channel conditions (ideal; trace at 15 dB; trace at 5 dB) and 10 seeds, for 720 runs of 100 rounds each with $K = 10$, $B = 3$, $E = 5$ and $\alpha = 0.5$. Within a seed, the data split, initial model, attacker identities and trace positions are identical across methods, so every comparison is paired. The runs were executed on an NVIDIA A100 GPU.

The analysis plan was fixed before any run:

- The *primary* quantity is the telemetry effect under stress, $[(\text{CCRA}-\text{mask-only})_{5\,\text{dB}} - (\text{CCRA} - \text{mask-only})_{15\,\text{dB}}]$, averaged over the three attacks within each seed.
- An effect counts as practically meaningful only if its 95% interval lies entirely beyond ± 1 percentage point.
- Intervals are percentile bootstraps over seeds with 10,000 resamples; rounds and attack cells are not treated as independent samples.
- A run that diverges numerically is scored at chance accuracy (10%) instead of being dropped, so unstable methods cannot look better by losing their worst runs.
- All other contrasts are exploratory and are not adjusted for multiple comparisons.

4.3 FedAvg Baseline Results

The first objective was a working FL pipeline without any defence. Ten UAVs trained TacticalEdgeCNN for 50 rounds with two local epochs, SGD (learning rate 0.01, momentum 0.9, batch size 64), on one BS with an ideal channel. Table 4.1 and Fig. 4.1 show the result.

Table 4.1 FedAvg baseline accuracy after 50 rounds

Data partition	Final test accuracy (%)	Best test accuracy (%)
IID (5,000 balanced images per UAV)	62.62	62.65
Non-IID, Dirichlet $\alpha = 0.5$	60.23	60.58

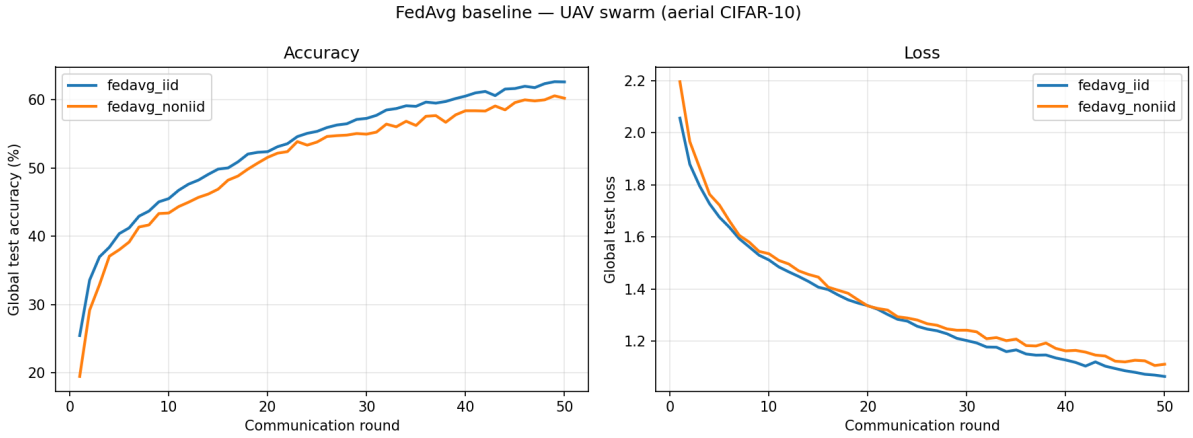


Figure 4.1 Test accuracy and loss of the FedAvg baseline over 50 rounds.

Non-IID data cost about 2.4 percentage points, and neither curve had fully levelled off by round 50. These numbers set the reference for the attack experiments: any defence should keep accuracy close to them when no attack is present.

4.4 Preliminary Study Results

The preliminary study compared FedAvg, CosAvg and an early version of CCRA with one BS, 30 rounds, one local epoch and a single seed. Three of the ten UAVs flipped their labels, and in the jammed scenarios a ground jammer targeted three honest UAVs. Table 4.2 lists the final accuracy (mean of the last five rounds), and Figs. 4.2 and 4.3 show the accuracy and the weight each method gave to the attackers.

Table 4.2 Preliminary study: final accuracy (%), single seed, 30 rounds

Scenario	FedAvg	CosAvg	CCRA (early)
No attack, ideal channel	46.5	**47.1**	44.5
No attack, jammed lossy channel	44.6	**45.6**	43.1
30% label flip, ideal channel	37.6	36.6	**43.3**
30% label flip, jammed lossy channel	36.8	39.7	**43.0**

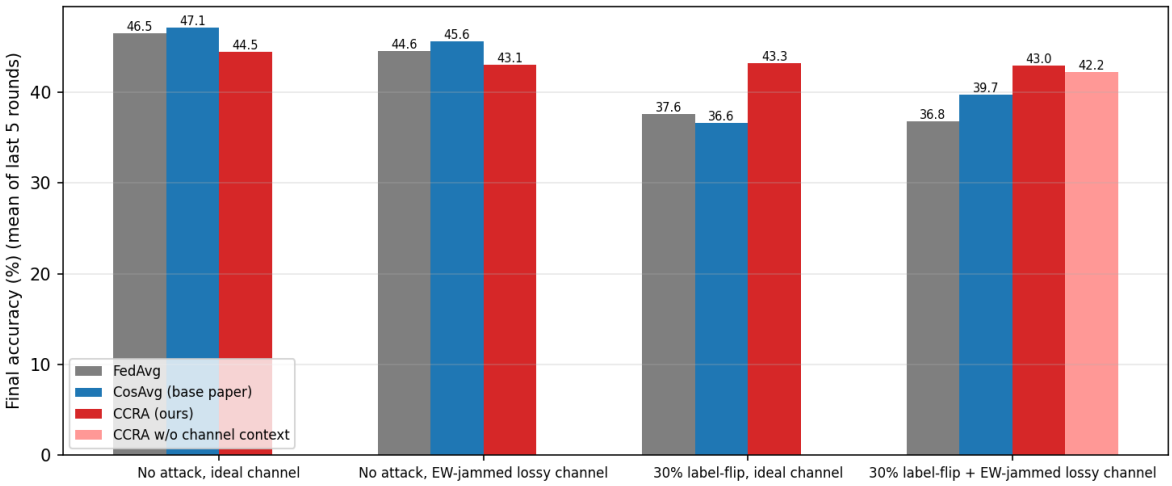


Figure 4.2 Preliminary study: final accuracy in the four scenarios.

The early CCRA kept about 43% accuracy in every attacked scenario and gave the attackers 1.9% of the aggregation weight on the ideal channel, against 19.1% for CosAvg and 26.0% for FedAvg. It cost about 2.5 points when there was no attack. These results were encouraging, but they could not support a claim:

- they came from one seed, so the size of the random variation was unknown;
- the channel-aware part of CCRA added only 0.8 points over its no-context ablation (42.2 to 43.0%), well within single-run noise;
- CCRA used a trusted root set while CosAvg did not, so the comparison mixed two differences;
- the reputation rule was revised when the system was extended to hierarchical FL, so the early CCRA is not the exact version evaluated in the controlled study.

These limitations defined the controlled study.

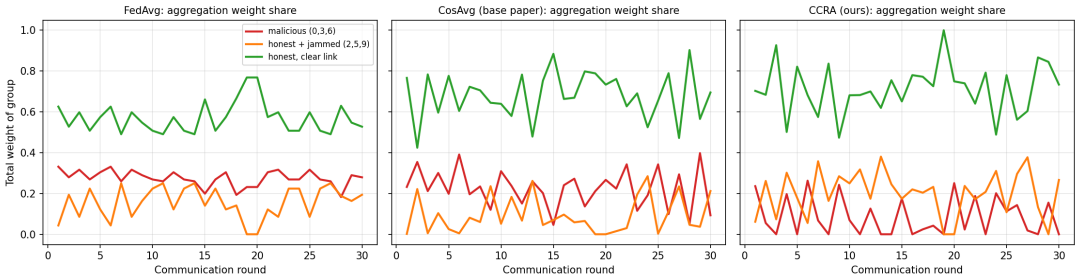


Figure 4.3 Preliminary study: aggregation weight given to attackers and to jammed honest UAVs under label flipping on the jammed channel.

4.5 Controlled Ablation Study Results

All 720 runs finished and were validated. The trace delivered on average 99.8% of packets at 15 dB and 65.0% at 5 dB.

4.5.1 Stability and Overall Accuracy

Table 4.3 summarizes every run, and Fig. 4.4 shows accuracy under attack across the three channel conditions.

Table 4.3 Outcomes over all 720 runs (120 per method). Diverged runs count as 10%.

Method	Diverged	All runs (%)	Attacked, 5 dB (%)
FedAvg	41/120	37.3	29.4
Median	26/120	29.3	21.3
FLTrust	0/120	50.6	**50.1**
No-context	0/120	51.4	45.3
Mask-only	0/120	**52.8**	49.5
CCRA (full)	0/120	51.1	46.0

FedAvg diverged in 41 and the coordinate-wise median in 26 of their 120 runs; the sign-flip attack destroys FedAvg in every seed. None of the methods that use a trusted reference update (FLTrust and the three CCRA variants) diverged in any run. The median does worse than expected because each cell holds only a few UAVs, so one attacker in a cell is already a large fraction of the values being compared. Over all runs the mask-only variant has the highest accuracy, 52.8%, against 37.3% for FedAvg. Table 4.4 breaks the result down by attack.

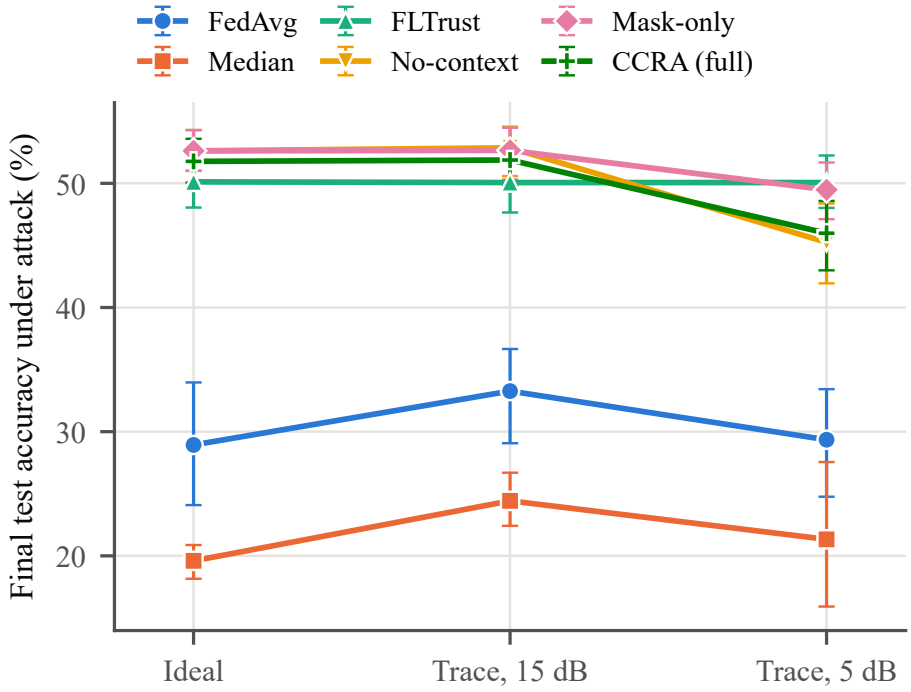


Figure 4.4 Final accuracy under attack (mean over sign flip, ALIE and IPM and 10 seeds), with 95% bootstrap confidence intervals.

Two points in Table 4.4 deserve attention. First, with no attack, plain FedAvg is the most accurate method on the ideal and 15 dB channels; every robust method pays a cost of roughly two to eight points for discarding or down-weighting honest updates. Second, on the 5 dB channel the masked variant is slightly ahead of FedAvg even without an attack, consistent with FedAvg's zero-filled lost coordinates now hurting it.

4.5.2 Which Kind of Channel Awareness Helps

Figure 4.5 shows the paired contrasts that separate the two channel mechanisms.

Erasure masking. On the ideal channel nothing is lost, so mask-only and no-context are identical. As the link weakens the masks matter more, and at 5 dB masking adds 4.2 points (95% CI [2.2, 5.9]). Treating lost coordinates as zeros shrinks honest updates, distorts their similarity scores and pulls the average towards zero; ignoring them avoids all three effects.

Telemetry in the reputation. The prespecified primary estimate, the telemetry effect at 5 dB minus that at 15 dB, is -2.7 points ($[-3.9, -1.5]$). The whole interval lies

Table 4.4 Mean final accuracy (%) by attack, method and channel. Best value in each block and column in bold.

Attack	Method	Ideal	Trace, 15 dB	Trace, 5 dB
No attack	FedAvg	**59.2**	**59.3**	54.1
	Median	52.5	52.7	49.7
	FLTrust	52.5	51.5	52.9
	No-context	57.1	56.7	51.1
	Mask-only	57.1	57.5	**54.7**
	CCRA (full)	55.4	56.6	52.6
Sign flip	FedAvg	10.0	10.0	10.0
	Median	10.0	19.6	12.4
	FLTrust	50.0	49.9	**50.6**
	No-context	**54.2**	**53.8**	46.1
	Mask-only	54.2	53.6	49.9
	CCRA (full)	52.5	52.8	46.5
ALIE	FedAvg	49.8	46.9	42.0
	Median	35.4	28.4	28.1
	FLTrust	49.9	50.6	49.6
	No-context	**52.9**	52.6	46.0
	Mask-only	52.9	**52.7**	**50.8**
	CCRA (full)	52.8	51.9	46.9
IPM	FedAvg	27.0	42.9	36.0
	Median	13.4	25.3	23.5
	FLTrust	50.5	49.7	**50.0**
	No-context	**50.7**	**52.3**	43.6
	Mask-only	50.7	51.7	47.7
	CCRA (full)	50.0	51.0	44.5

below the -1 point threshold, so by the rule fixed in advance the telemetry mechanism has a meaningful *negative* effect that grows as the link weakens. In absolute terms, at 5 dB CCRA minus mask-only is -3.5 points ($[-5.1, -2.1]$): 46.0% against 49.5%.

Both together. Full CCRA against no-context combines a gain and a loss; at 5 dB the net effect is small and its interval includes the equivalence region. The earlier impression that "channel context helps" came from this combination, which hid two opposite effects.

4.5.3 Mechanism: Who the Gate Rejects

Section 3.4.4 predicted that slowing rewards on weak links would push honest UAVs' reputations down without doing much to attackers. Figure 4.6 confirms that pattern.

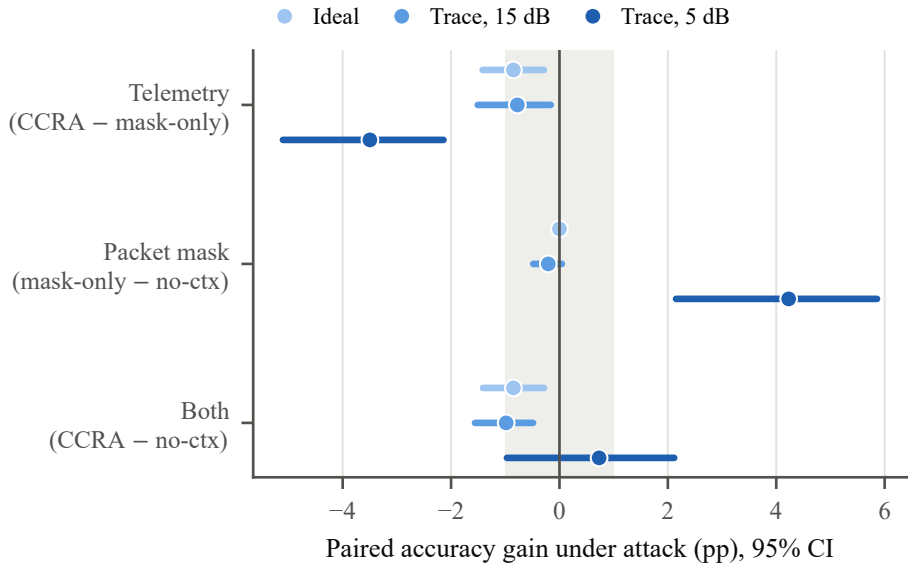


Figure 4.5 Paired accuracy differences under attack with 95% bootstrap intervals. The grey band is the prespecified ±1 point region of practical equivalence.

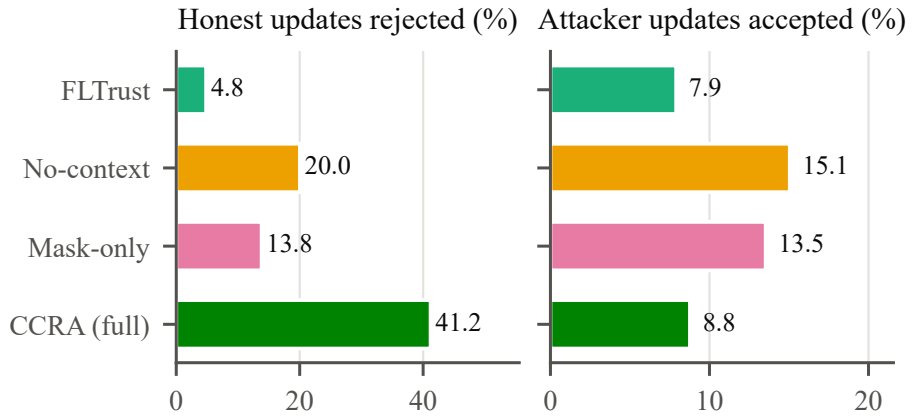


Figure 4.6 Gate decisions under attack at 5 dB, as a percentage of received updates, averaged over attacks and seeds.

With telemetry, the gate rejected 41.2% of honest updates against 13.8% without it, while the share of attacker updates accepted fell only from 13.5% to 8.8%. The rule gave up a large amount of honest data to filter slightly more attacker data, and it did so precisely when honest data were already scarce because of packet loss. These rates are descriptive. They agree with the analysis in Section 3.4.4, but they do not by themselves prove that it is the cause.

4.5.4 Comparison with FLTrust

FLTrust is the closest established method, since it uses the same root set. Its gate rejected only 4.8% of honest updates at 5 dB, because it has no memory: each round is judged on its own. In an exploratory paired comparison, mask-only was 2.6 points better than FLTrust under attack at 15 dB ([0.9, 4.5]), but at 5 dB the two were indistinguishable (-0.6 points, $[-2.0, 1.0]$). The two methods differ in more than memory, so this comparison cannot isolate its effect; one reading consistent with the mechanism above is that reputation memory helps on good links and stops helping on weak ones.

4.6 Discussion

4.6.1 What the Results Show

1. A trusted reference update is what keeps training stable: every method that used one avoided divergence, and every method that did not diverged in some runs.

2. Among channel-aware mechanisms, erasure masking is simple, costs nothing on a good channel (its 15 dB contrast is inside the equivalence region) and helps substantially on a weak one.

3. The intuitive idea of letting link quality soften judgement backfired. The failure is structural: an asymmetric reputation update biases trust downwards for exactly the UAVs it was meant to protect.

4.6.2 Why a Negative Result Is Useful

The preliminary study suggested that CCRA's channel context was its key ingredient. Had that claim been carried forward, Phase II would have been built on a component that harms accuracy. The prespecified design, paired seeds and chance scoring of diverged runs made it possible to reject the claim with confidence, and the analysis in Section 3.4.4 points to the specific rule that must change.

4.6.3 Threats to Validity

- **Channel realism.** The trace comes from one measured BS; the other BSs are virtual, and the 5 and 15 dB anchors are assumptions, not calibrated values. The runs contain few handovers, so no claim is made about high mobility.
- **Scope of learning task.** One dataset, one model, ten UAVs and fixed attack strengths were used.
- **Attacker model.** Telemetry is assumed authentic, and adaptive attacks on the reputation itself [Liu et al., 2026] were not tested.
- **Statistics.** Ten seeds give approximate intervals; secondary contrasts are not adjusted for multiple comparisons. Any revised rule must be tested on new seeds, not re-tuned on these ten.

Chapter 5

Conclusions and Future Scope of Study

5.1 Summary of the Work

This project built a simulation framework for federated learning over a UAV swarm whose updates travel over lossy air-to-ground links and in which some UAVs are Byzantine. The framework supports hierarchical aggregation at three base stations, handover, packet erasures from a synthetic jammed channel or a measured UAV trace, five attacks and eight aggregation rules, and is covered by automated tests. A FedAvg baseline was established first. A reputation-based aggregator, CCRA, was then designed with two forms of channel awareness, erasure masking and telemetry-weighted reputation. After a promising but inconclusive single-seed study, a prespecified, paired study of 720 runs separated the two forms and measured each one's effect. The results have been written up as a manuscript in IEEE letter format.

5.2 Conclusions

1. Non-IID data reduced FedAvg accuracy from 62.6% to 60.2% in the baseline, and a 30% label-flip attack reduced it by about ten points in the preliminary study.

2. Aggregators that judge updates against a trusted reference update never diverged in 720 runs, whereas FedAvg and the coordinate-wise median diverged in 41 and 26 of 120 runs each.

3. Ignoring lost coordinates instead of treating them as zeros (erasure masking) im-

proved accuracy under attack on a 5 dB link by 4.2 percentage points (95% CI [2.2, 5.9]).

4. Using link telemetry to slow reputation rewards had a meaningful negative effect: the prespecified primary estimate was -2.7 points ($[-3.9, -1.5]$), and the gate rejected 41.2% of honest updates instead of 13.8%.

5. The likely cause is structural. With slowed rewards but full-rate penalties, the fixed-point condition (3.9) shows that an honest UAV's reputation settles towards its worst rounds rather than its average behaviour, and the measured gate decisions match this prediction.

6. On weak links, the best variant is statistically indistinguishable from FLTrust (an exploratory comparison), so its advantage over FLTrust appears only on ideal and moderate channels.

5.3 Scope for Future Work (Phase II)

1. **Redesign the reward rule** so that weak links do not bias reputation downwards, for example by making the update symmetric in the evidence, or by normalizing each UAV's score against its own recent history, and freeze the rule before testing.

2. **Confirm on fresh seeds and conditions.** Evaluate the revised rule on new seeds and channel settings rather than the ten used here, so that tuning cannot inflate the result.

3. **Mobility and handover.** Use traces or mobility models with frequent handovers, and more base stations, to test whether telemetry helps when the dominant impairment is handover rather than fading.

4. **Stronger attackers.** Add adaptive attacks that target the reputation itself and attackers that spoof their link telemetry.

5. **Network-level simulation.** Replace the packet-error model with a network simulator such as NS-3 to capture medium access, retransmissions and latency.

6. **Generalization.** Repeat the key comparisons with another dataset, a larger model and a larger swarm, and submit the manuscript.

References

A. Alkhateeb, G. Charan, T. Osman, A. Hredzak, J. Morais, U. Demirhan, and N. Srinivas. DeepSense 6G: A large-scale real-world multi-modal sensing and communication dataset. *IEEE Commun. Mag.*, 61(9):122–128, 2023. doi: 10.1109/MCOM.006.2200730.

S. Barkatsa, M. Diamanti, P. Charatsaris, E. E. Tsiropoulou, and S. Papavassiliou. Jamming attack mitigation in wireless federated learning networks using Bayesian games. *IEEE Netw. Lett.*, 6(4):247–251, 2024. doi: 10.1109/LNET.2024.3499360.

M. Baruch, G. Baruch, and Y. Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.

P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

X. Cao, M. Fang, J. Liu, and N. Z. Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2021. doi: 10.14722/ndss.2021.24434.

G. Charan and A. Alkhateeb. Sensing-aided 6G drone communications: Real-world datasets and demonstration. *IEEE Open J. Commun. Soc.*, 2025. doi: 10.1109/OJCOMS.2025.3612320. early access.

T.-M. H. Hsu, H. Qi, and M. Brown. Measuring the effects of non-identical data distribution for federated visual classification. arXiv:1909.06335, 2019.

M. Li, D. Xiao, J. Liang, and H. Huang. Communication-efficient and Byzantine-robust differentially private federated learning. *IEEE Commun. Lett.*, 26(8):1725–1729, 2022. doi: 10.1109/LCOMM.2022.3180113.

J. Liao, B. Jiang, P. Zhao, L. Ning, and L. Chen. Unmanned aerial vehicle-assisted federated learning method based on a trusted execution environment. *Electronics*, 12 (18):3938, 2023. doi: 10.3390/electronics12183938.

X. Liu, E. Liu, W. Ni, H. Zhang, Z. Wei, and E. Hossain. Learning to evade: Adaptive and stealthy attack on reputation-based federated learning. *IEEE Commun. Lett.*, 30: 1836–1840, 2026. doi: 10.1109/LCOMM.2026.3685107.

B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, volume 54 of *PMLR*, pages 1273–1282, 2017.

M. Shirvanimoghaddam, A. Salari, Y. Gao, and A. Guha. Federated learning with erroneous communication links. *IEEE Commun. Lett.*, 26(6):1293–1297, 2022. doi: 10.1109/LCOMM.2022.3167094.

Y. Xiao, Y. Ye, S. Huang, L. Hao, Z. Ma, M. Xiao, S. Mumtaz, and O. A. Dobre. Fully decentralized federated learning-based on-board mission for UAV swarm system. *IEEE Commun. Lett.*, 25(10):3296–3300, 2021. doi: 10.1109/LCOMM.2021.3095362.

C. Xie, O. Koyejo, and I. Gupta. Fall of empires: Breaking Byzantine-tolerant SGD by inner product manipulation. In *Proc. Conf. Uncertainty Artif. Intell. (UAI)*, volume 115 of *PMLR*, pages 261–270, 2019.

D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proc. Int. Conf. Mach. Learn. (ICML)*, volume 80 of *PMLR*, pages 5650–5659, 2018.

Experimental Settings

Table A.1 Settings of the controlled study

Parameter	Value
UAVs K, base stations B	10, 3
Byzantine UAVs f	3, fixed per seed across all methods
Rounds; synchronization period E	100; 5
Local training	1 epoch, SGD, learning rate 0.01, momentum 0.9, batch 64, weight decay 5×10^{-4}
Model	TacticalEdgeCNN, about 156k parameters, group normalization
Data	Aerial-transformed CIFAR-10, Dirichlet $\alpha = 0.5$, 49,500 training images to UAVs
Root set $\mathcal{D}_0$	500 images (50 per class), disjoint from all UAV data
Reputation	$\beta = 0.5$, gate $\tau = 0.3$, initial trust 1
Staleness discount	$\exp(-0.5 \cdot \text{staleness})$; late updates dropped
Packets	$L = 1024$ parameters, 32 bits per parameter
Packet errors	logistic, slope $a = 1$, threshold $\gamma_{\text{th}} = 3\,\text{dB}$
Round deadline; bandwidth	2 s; 20 MHz
Handover	3 dB hysteresis, 2-round time to trigger
Trace	DeepSense 6G Scenario 23, stride 5 samples per round, segments ≥ 200 samples
Reference SNR $\bar{\gamma}$	15 dB (good link), 5 dB (weak link)
Attacks	sign flip $\lambda = 5$; ALIE-style $z = 1.5$; IPM $\epsilon = 0.5$
Seeds	41–50

Reproducing the Results

All code, the run-level results and this report are in the repository https://github.com/sasank-555/uav-swarm-fl.

Table B.1 Repository layout

Path	Contents
`fl/`	Simulator: data, model, client, channel, topology, attacks, aggregators, server
`tests/`	43 automated tests
`run_context_study.py`	Plans and runs the 720-run controlled study
`analyze_context_study.py`	Prespecified analysis with paired bootstrap
`results/context_study/`	Run-level CSV and analysis of the controlled study
`scripts/make_paper_figures.py`	Builds every figure, table and number in the paper and this report
`paper/`	Manuscript in IEEE letter format
`report/`	This report

The main steps are:

1. Install the dependencies: `pip install -r requirements.txt`.
2. Prepare the data: `python prepare_data.py`, and convert the DeepSense Scenario 23 download with `scripts/prepare_deepsense23.py`.
3. Run the study (about 720 GPU runs): `python run_context_study.py -profile final -device cuda -parallel 4`.
4. Analyse it: `python analyze_context_study.py results/context_final`.
5. Rebuild all figures and numbers: `python scripts/make_paper_figures.py`.

Step 5 alone is enough to regenerate this report's figures from the committed results.